

AGI CLUB · SÃO PAULO · 2026

Minimal Oversight

Quanta revisão os agentes de IA precisam — e onde?

Um sistema, um framework, 150.000 traces reais de execuções

Carlos R. B. Azevedo · Senior Director of AI Engineering · github.com/crbazevedo · [arXiv:2606.15563](https://arxiv.org/abs/2606.15563)



Carlos R. B. Azevedo

Pesquisador e engenheiro de IA · São Paulo

- Doutor em engenharia de computação. Pesquisa sobre **tomada de decisão sob incerteza**: otimização multiobjetivo, aprendizado antecipatório e simulação estocástica.
- Liderou equipes de IA e ciência de dados na indústria e em consultoria.
- Escreveu a teoria **Minimal Oversight** (arXiv:2606.15563) e desenvolve o **delegation-lab** abertamente.

→ **Cada afirmação desta noite remete a um paper, um lab ou uma execução registrada.**

O que sustenta esta noite

O PAPER

Minimal Oversight

A teoria: a menor revisão que mantém a qualidade acima da meta.

arXiv:2606.15563 · público

O LAB

delegation-lab

Código executável: as equações, os fluxos de trabalho e o harness das execuções ao vivo.

github.com/crbazevedo/delegation-lab

`pip install minimal-oversight`

OS DADOS

Um log de produção

Dois meses de entrega multiagente real, desidentificados.

150.000 eventos

→ O paper e o lab são públicos. Tudo mostrado esta noite pode ser reexecutado a partir deles.

Dois meses de trabalho de agentes. Uma pessoa lendo tudo.

877

TAREFAS, ISSUE → MERGE

150.000

EVENTOS REGISTRADOS

1

REVISOR

Ler tudo, a 5 minutos por tarefa: **9 dias úteis.**

Quanta revisão?

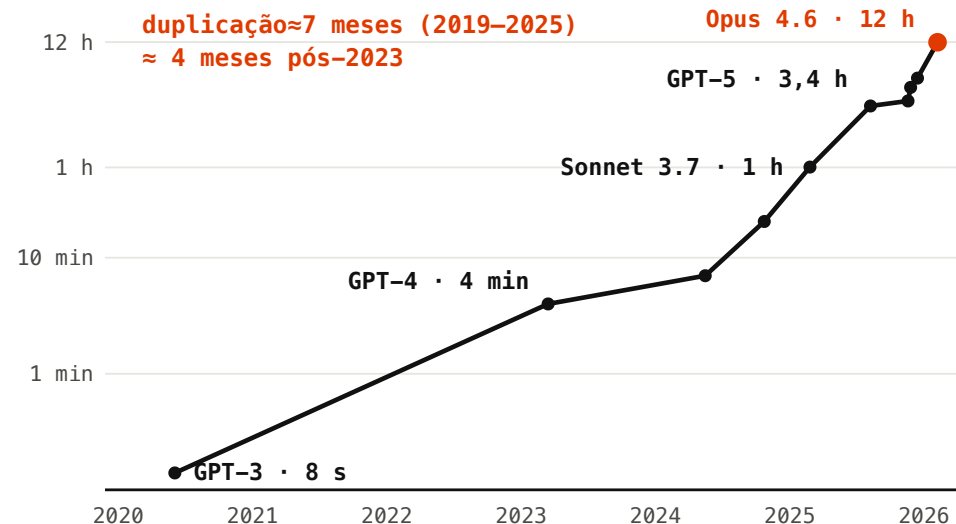
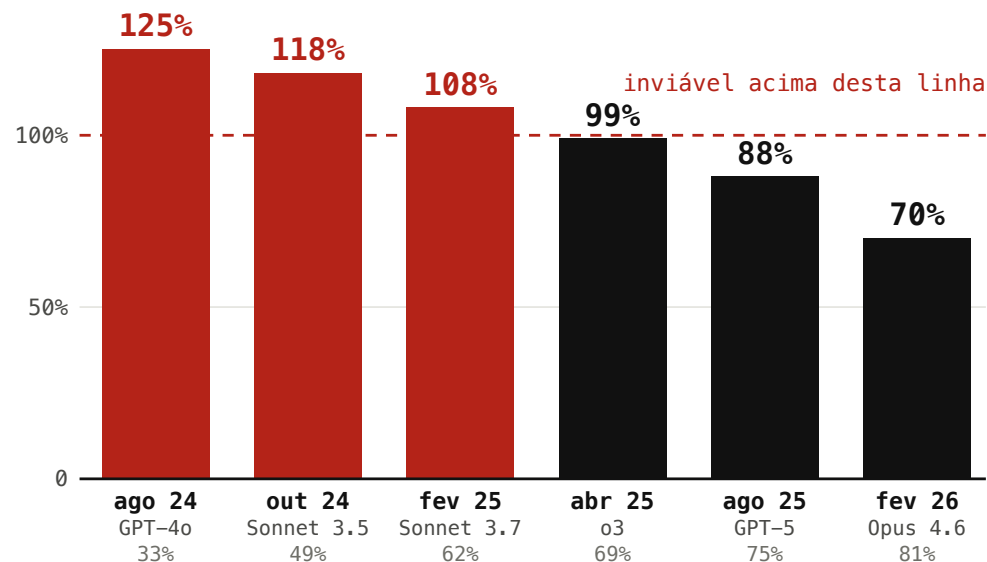
Onde no workflow?

Quando a autonomia é impossível?

A necessidade muda de forma desde 2020

cobertura necessária para uma meta de 90% · calculada
modelos de pesos abertos cruzam depois: Kimi K2 Thinking, nov 25, precisa de

duração de tarefa concluída com 50% de confiabilidade · METR 1.1



A alucinação caiu dentro de cada instrumento fixo: sumarização fundamentada passou de **3,0%** (GPT-4, 2023) para cerca de **1%** (2025); o conjunto de dados mais difícil de nov 2025 reposicionou a fronteira entre **2 e 15%**. A lembrança em domínio aberto sem navegação ainda erra perto de **40%**.

→ **A revisão por tarefa caiu; a supervisão migrou para o sistema.**

calculado: identidade de viabilidade (c 0,80 · f 0,85) sobre SWE-bench Verified (retirado em fev 2026) · METR 1.1 · HHEM

Três respostas que todo mundo tenta primeiro

RESPOSTA 1

“Revisar tudo uniformemente.”

Parece seguro. A frota supera um leitor só.

RESPOSTA 2

“Confiar no ci verde.”

Mergeou limpo, então o agente deve ser bom.

RESPOSTA 3

“Vigiar o trabalho mais fraco.”

Olhos seniores nas mudanças mais arriscadas.

I	O Sistema	cada ação deixa registro
II	A Teoria	a aritmética da revisão
III	O Teste	quatro previsões contra os dados
IV	A Prática	execuções ao vivo + os hábitos

→ **As três respostas falham esta noite contra a teoria e 150.000 rastros reais.**



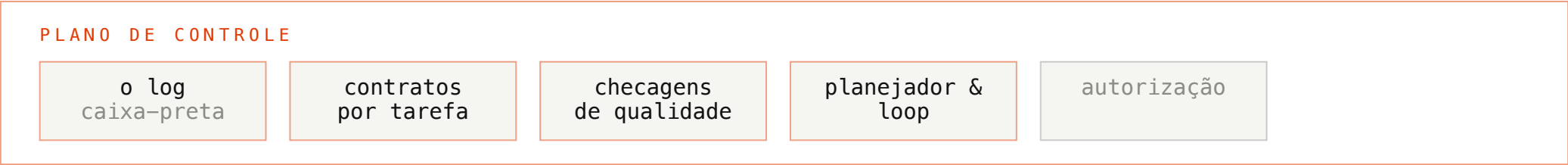
0 SISTEMA

O que fica registrado quando agentes trabalham?

Quatro partes de um plano de controle, uma tarefa de exemplo.

Um plano de controle acima de todos os agentes

Modelo os pesos	Agente modelo + ferramentas, em loop	Harness Claude Code, Codex CLI	Plano de controle planeja · despacha · checa · audita
---------------------------	--	--	---

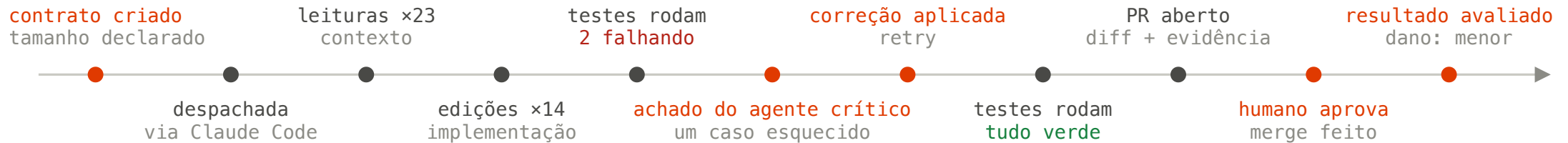


admissionno loop contratos ↓ eventos ↑ harness A (Claude Code)sessão · sessão · sessão harness B (Codex)sessão · sessão agentes próprios de SDLCvia adaptadores PLANO DE DADOS

repositório compartilhado · CI · artefatos · checagens em cada merge

NOSSO EXEMPLO · “burst-guard” : impedir que um tenant barulhento derrube a API para todos.

Uma tarefa, um log: tudo que “burst-guard” fez



- Append-only: cada evento tipado, com timestamp, atribuído.
Dashboards são computados a partir dele.
- Se não está no log, não aconteceu.
- Um incidente dá replay passo a passo, uma semana depois.

→ Este log é o conjunto de dados do Ato III.

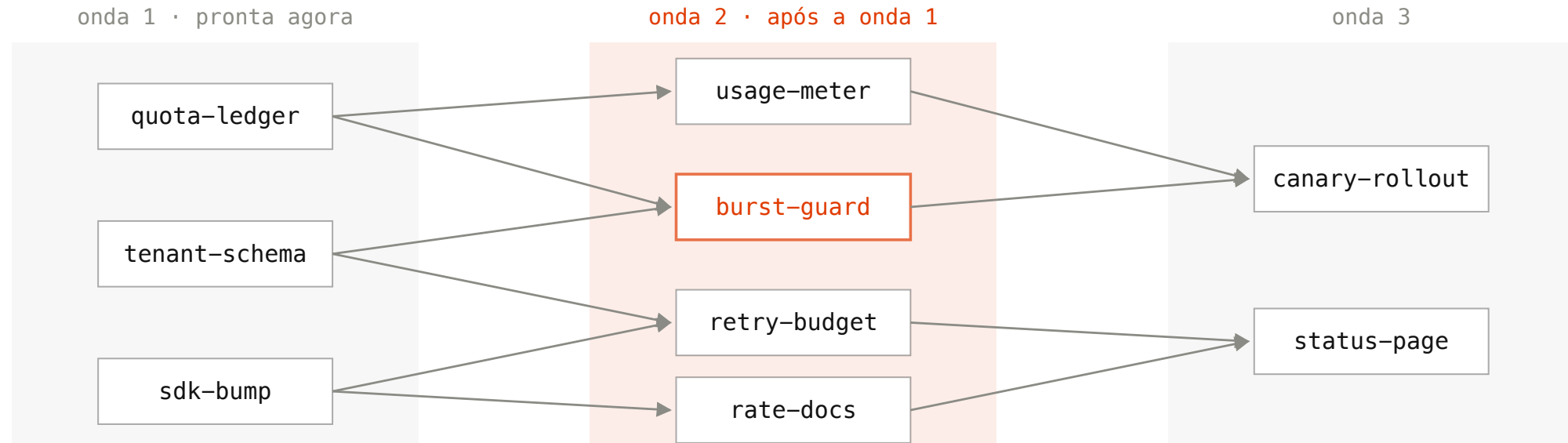
Toda tarefa começa como um contrato escrito

```
 tarefa: burst-guard
 objetivo: rate limiting por tenant na API
 pública
 pronto quando: 429 ao estourar · config por
 tenant · docs
 pode ler: api/gateway/** · docs/limits.md
 pode escrever: api/gateway/limits/**
 autonomia: implementa + abre PR · merge segue
 humano
 tamanho declarado: M
```

- O prompt é um ponteiro para este contrato.
- Ele não muda enquanto a tarefa roda.
- Áreas de escrita nunca se sobrepõem; agentes paralelos não colidem.
- **O tamanho é declarado antes de existir código.**

→ **Fixado de antemão: o que foi pedido, a quem, com quais limites.**

O trabalho forma um grafo de dependências



- Uma onda: toda tarefa com insumos prontos, tudo de uma vez.
- O paralelismo esbarra em escrita sobreposta e capacidade de revisão. **Computação nunca limita.**

→ O grafo de dependências limita a velocidade do trabalho.

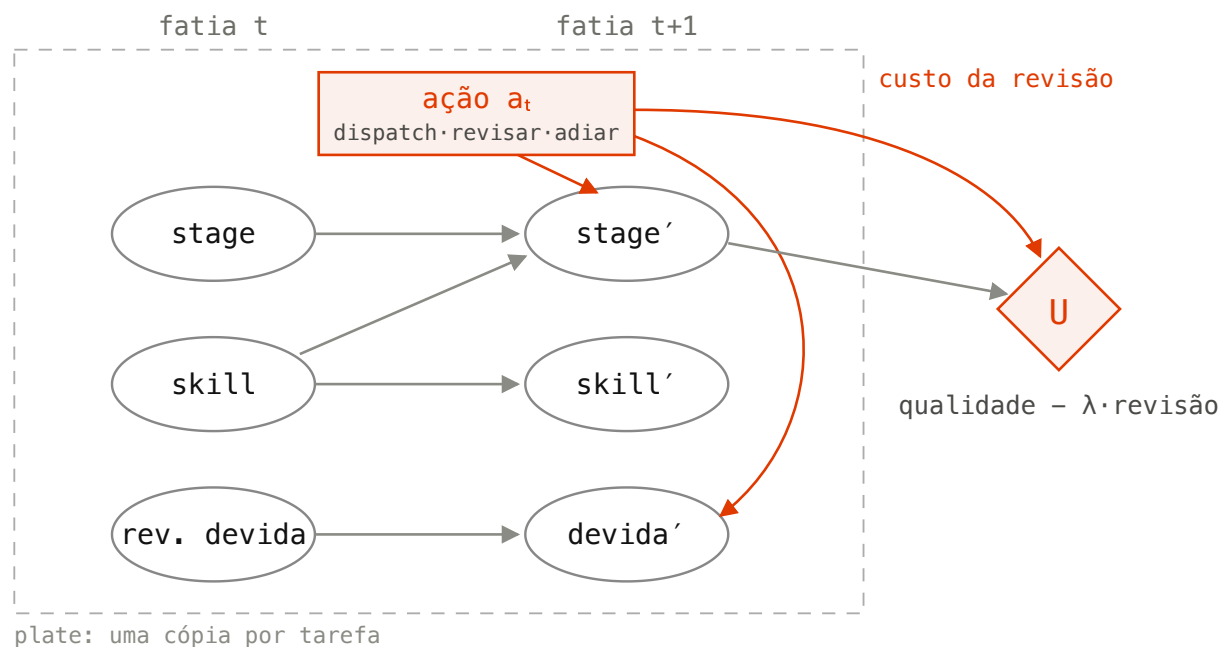
Escolher o que roda quando é um problema de decisão

```
// o mundo do planejador, em RDDDL
stage(t) : {intake, build, ship, done}; // variáveis
por tarefa
skill(t) : real; dispatch(t), review(t) : action;
stage'(t) = if dispatch(t) // despacho: uma aposta com
probabilidades conhecidas
    then Discrete(done: skill(t), build: 1-skill(t)) else
stage(t);
// atualização do lab: skill 0.78→0.70 ⇒ coverage
0.90→1.00
reward =  $\sum_t$  [ quality(t) -  $\lambda$ ·review(t) ]; // a tese
```

- O estado é um punhado de variáveis por tarefa e por agente: **stage**, **tamanho declarado**, **habilidade estimada**, **revisão devida**.
- O próximo valor de cada variável depende de poucos pais. A recompensa soma por tarefa.
- Um solver devolve uma **política**: para cada estado, a próxima ação, recomputada a cada onda.
- Resultados fora da previsão alimentam a próxima onda por construção.

→ O grafo e os escopos de escrita são restrições duras. **A política escolhe dentro delas.**

O modelo de transição, desenhado como rede de Bayes



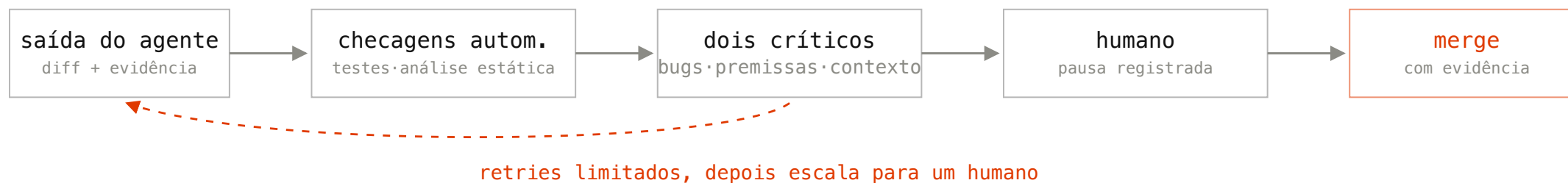
- Duas fatias de tempo. Uma seta por dependência.
- Um resultado revisado atualiza a **skill**; a skill reprecifica as tarefas abertas do agente.
- **Revisão devida não tem seta para skill:** backlog não diz nada sobre qualidade.
- Poucos pais por variável: tabelas pequenas, planejamento barato.

$$P(s' \mid s, a) = \prod_i P(s_i' \mid \text{parents}(s_i'), a)$$

- **Desenrolada para frente, planeja a onda. Condicionada no log, estima habilidade e deriva.**

→ Um diagrama, duas direções: **prevê para frente, aprende para trás.**

Três camadas de checagens, depois um humano



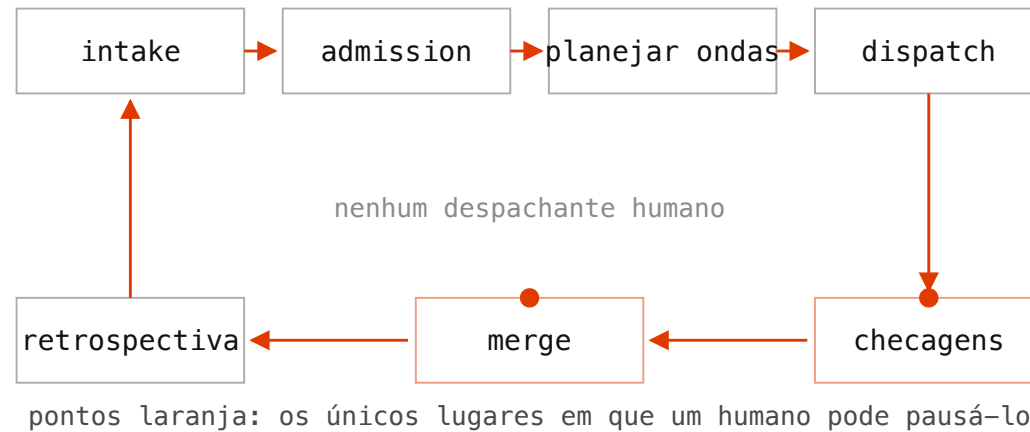
checagens testes · estática · CI → passou

crítico “limites são por tenant; os testes só cobrem o tenant padrão” → detectado

retry testes de tenant adicionados → verde · **humano** aprova em 30 s → merge feito

→ Cada erro encontrado vira **uma nova checagem automática**.

O loop que roda sozinho



54+ h

EXECUÇÃO MAIS LONGA, TOTALMENTE AUTÔNOMA

- **Intake** lê os roadmaps e cria tarefas candidatas que já nascem como contratos.
- **Admission** controla cada candidata: dependências, áreas de escrita, capacidade e política. Veredictos: **aceitar** · **reduzir** · **endurecer** · **adiar** · **rejeitar**.
- Cada handoff vira regra. Humanos mantêm os pontos de pausa.
- O loop pode se pausar. Nunca pode parar em silêncio.

→ O roadmap dá intenção. As checagens dão confiança. **Ninguém vigia o loop.**

Cinco números por tarefa, e uma surpresa

NÚMERO	MEDIDO EM
feito certo, antes da revisão	primeira passada
feito certo, depois da revisão	merge
tamanho declarado	planejamento
revisão recebida	checagens & pausas
dano quando falhou	retrospectiva

**O tempo desse revisor é bem gasto?
Isso exige um frame teórico.**

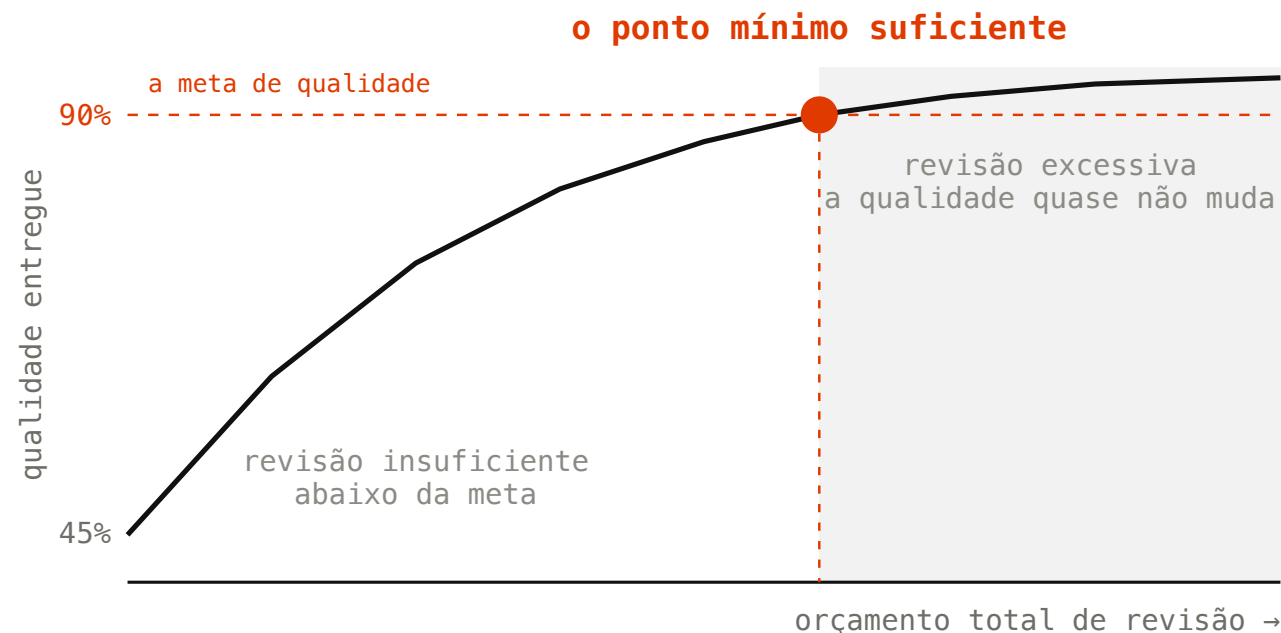
II

A TEORIA

Qual é a **menor revisão** que mantém a qualidade?

Supervisão mínima suficiente (MSO). Uma fórmula, usada de quatro
jeitos. arXiv:2606.15563.

Supervisão mínima suficiente, em uma figura



Suficiente: qualidade entregue $\geq$ meta.

Mínima: nem uma revisão a mais.

- A revisão tem retornos decrescentes: os primeiros olhares compram mais.

→ Gaste revisão até o ponto e então pare.

O que a revisão faz com a qualidade

$$\text{depois} = \text{antes} + (1 - \text{antes}) \times \text{olhou} \times \text{detectou} \times \text{corrigiu}$$

o time de burst-guard, toda tarefa revisada: $0.45 + 0.55 \times 0.80 \times 0.85$
 ≈ 0.82

OLHOU

fração das tarefas que você revisa

DETECTOU

erros que você nota quando olha

CORRIGIU

erros notados de fato reparados

→ Três botões, todos mensuráveis a partir dos cinco números do log.

O dashboard diz 82%. Qual agente é esse?

HABILIDADE PRÉ-REVISÃO	QUALIDADE DA REVISÃO	DASHBOARD
82%	nenhuma necessária	82%
45%	detecta 80% · corrige 85%	82%
28%	detecta 86% · corrige 87%	82%

82 ÷ 45 ≈ **1.8x de bajulação**, medida em produção.

+36%

AUTONOMIA CONCEDIDA A MAIS VIA DASHBOARD

31×

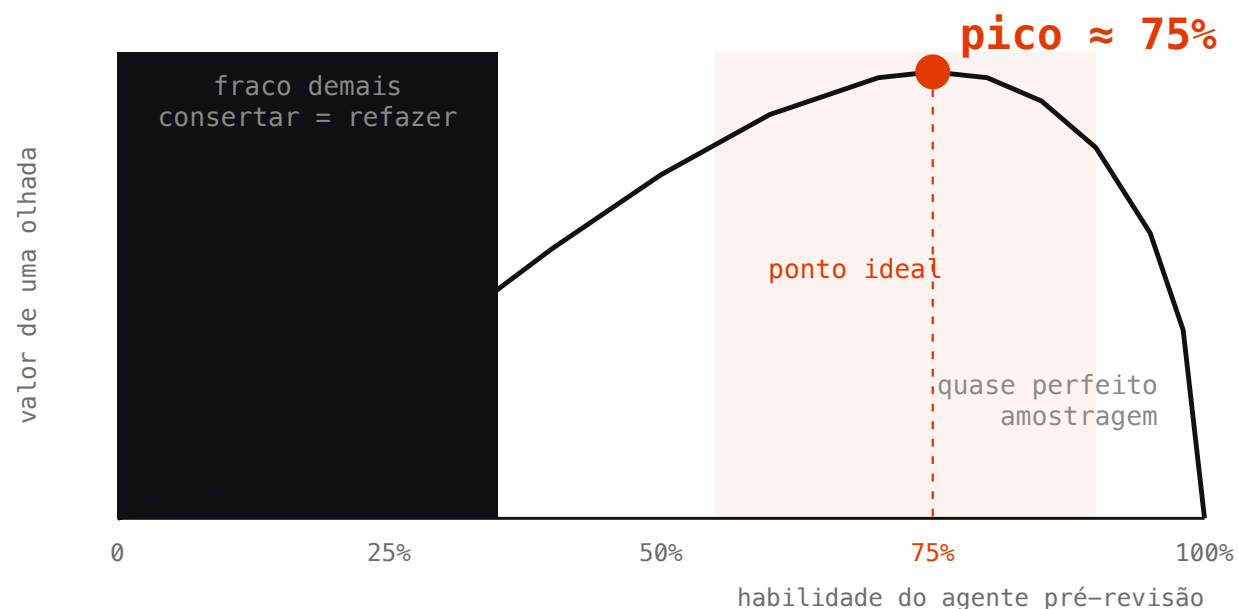
ERRO POSSÍVEL DA ESTIMATIVA DE HABILIDADE

Registre **dois resultados por tarefa**: antes da revisão, depois da revisão.

× “Confiar no pipeline verde”

→ O verde mede o workflow. Autonomia precisa do número pré-revisão.

A revisão paga mais em agentes quase bons



- **Agente fraco:** reparar é refazer. Realoque.
- **Agente quase perfeito:** uma olhada raramente muda algo. Amostre.
- **No meio:** uma olhada costuma virar falha → passou. Gaste aqui.

× “Vigiar o trabalho mais fraco”

97%	88%	74%	62%	41%	22%
amostragem	leve	mais revisão	pesada	encolha a tarefa	não delegue

→ Gaste cada olhada **onde ela compra mais qualidade.**

Algumas metas são impossíveis em qualquer orçamento

meta 90% · agente em 45% · detecta 80% · corrige 85%

lacuna a fechar: $0.90 - 0.45 = 0.45$

cada revisão recupera: $0.55 \times 0.80 \times 0.85 = 0.374$

$0.45 \div 0.374 = 1.20 \rightarrow$ **revisar 120% das tarefas**

× **impossível: revisão sozinha não fecha a lacuna**

meta de 90% = cobertura de 94%

QUANDO A MATEMÁTICA DIZ NÃO

- Encolher as tarefas
- Testes & críticos melhores
- Loop de reparo melhor
- Baixar a meta, ou manter um humano

Todo pipeline tem um **teto de qualidade**: metas acima dele falham em qualquer orçamento.

Mais peças móveis $\rightarrow$ **autonomia segura mais curta**.

$\rightarrow$ “Isso pode rodar sozinho?” se resolve com uma divisão.

A forma do workflow muda a matemática



→ A olhada de maior alavancagem num grafo fica no merge.

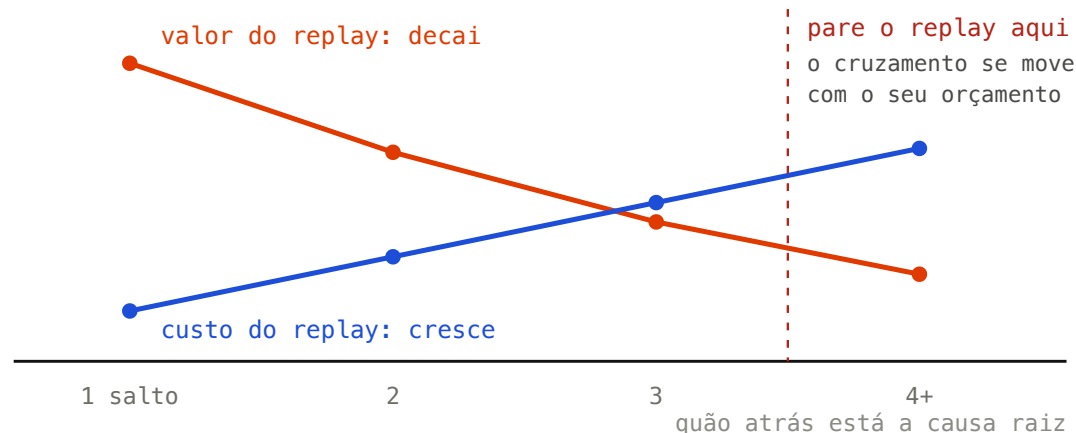
Cobertura se paga em toda tarefa; replay, só na captura.

Sinalizar e seguir · grátis
quando a precisão do crítico é baixa

Patch na saída · barato
fraco em defeitos herdados

Retry do passo · médio
funciona onde o defeito nasceu

Replay da raiz · caro
a única cura para cascatas



além de ≈ 6 saltos, replay perde até com orçamento ilimitado

- A cada salto para trás, mais um estágio precisa sobreviver de novo: o valor cai, a conta sobe.

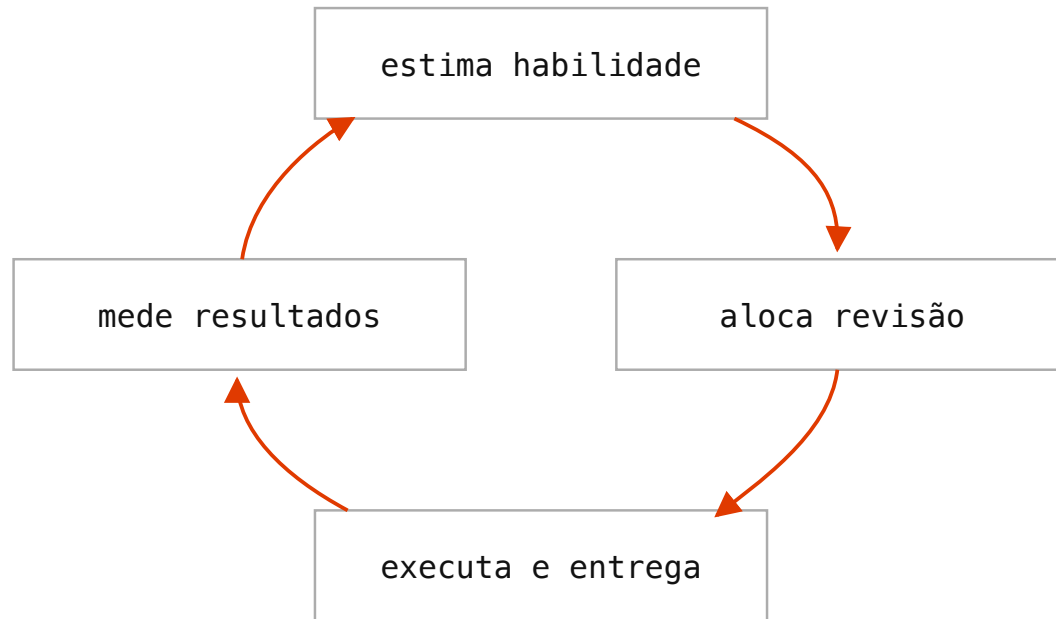
GATE	QUALIDADE POR TOKEN
build	0.047
ship	0.043

intake **0.013 · um terço**

A melhor política financia build e ship, **pula intake**, e paga os replays.

→ Um pipeline saudável dá replay de propósito. Replays subindo apontam o gate subfinanciado.

Habilidades derivam. Planos de revisão vencem.



5

UPDATES DE MODELO EM DOIS MESES · HABILIDADES
MEXERAM TODA VEZ

Solte revisão onde a habilidade subiu.

- Reestime em agenda fixa.
- Mantenha uma pequena auditoria aleatória sempre ligada.

→ **A alocação se repete, porque as habilidades seguem mudando.**

III

0 TESTE

Quatro previsões, checadas contra os dados.

- 1 · Fluxos de trabalho bajulam modelos
- 2 · Pedidos maiores falham mais
- 3 · A revisão fica onde a falha não está
- 4 · Planos fixos envelhecem; adaptativos seguram

dados: o log de 150.000 eventos + SWE-bench Verified

Dois conjuntos de dados, três métodos

LOG DE PRODUÇÃO

793

tarefas concluídas, de 877 abertas. Revisão e governança reais; sem gabarito.

SWE-BENCH VERIFIED

500 × 16

tarefas × modelos, testes ocultos. Com gabarito; sem governança.

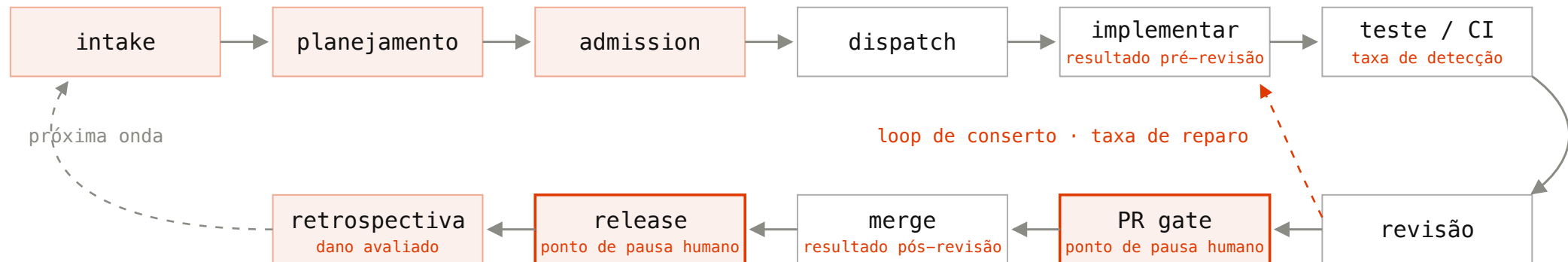
Fluxo de trabalho recuperado do log por mineração de processos: 16 papéis achados.

Dano avaliado em 444 falhas por juízes LLM: concordância 0.84, tudo dentro de um nível.

Simulador calibrado em dados reservados: o modelo do planejador do Ato I, condicionado nos eventos.

→ **Traces dizem o que aconteceu. O simulador diz o que teria acontecido.**

O workflow, recuperado do log



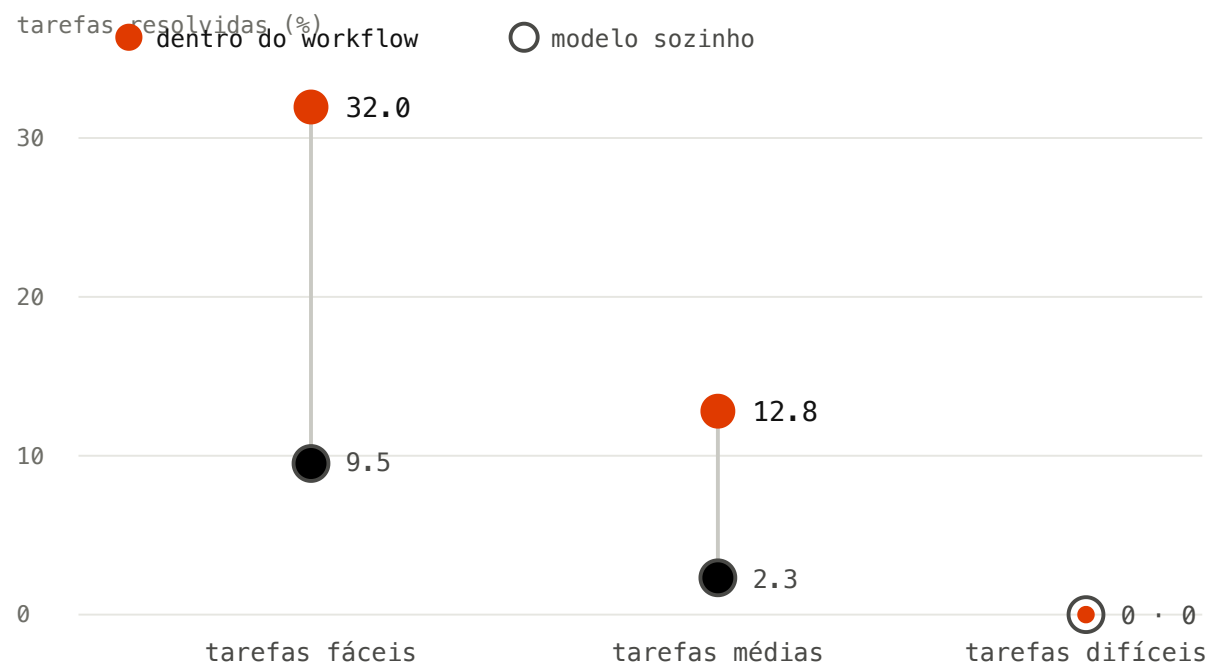
caixas vermelhas: governança · borda grossa: pausa humana

→ Cada achado a seguir aponta de volta para este mapa.

PREVISTO fluxos de trabalho fazem modelos parecerem melhores do que são

CONFIRMADO · MEDIDO · CAUSALIDADE EM ABERTO

O mesmo modelo parece 4x melhor dentro do workflow



3.9x

WORKFLOW VS. MODELO SOZINHO, MODELOS PAREADOS

73–89%

DAS VITÓRIAS DO WORKFLOW: TAREFAS QUE O MODELO FALHOU SOZINHO

Em produção este mecanismo mediu **1.8x**.
Nas tarefas mais difíceis a diferença some.

→ **Conceda autonomia pela habilidade solo; resultado de workflow bajula o modelo.**

PREVISTO pedidos maiores falham mais, mesmo em dificuldade igual

CONFIRMADO · CORRELAÇÃO

Pedidos maiores falham mais. Mais difíceis, bem menos.

dano médio em 444 tarefas falhas · mais escuro = pior

tamanho do pedido	grande	1.69	1.39	1.69	média da lin 1.62
	médio	1.27	1.48	1.39	1.35
	pequeno	0.86	0.93	1.48	1.07
		baixa	média	alta	
		dificuldade			

- Tamanho e dificuldade quase não se correlacionam aqui.

Um degrau a mais de **tamanho** $\approx 2\times$ o dano de um degrau de dificuldade.

- Igual no SWE-bench: patches maiores, menos resolvidos.

Se uma tarefa pede dois rótulos de tamanho, **divida**.

→ O tamanho é fixado no planejamento, antes de existir código.

PREVISTO a revisão humana fica onde a falha não está

CONFIRMADO · OBSERVADO

A revisão guardou o fim. Os erros nasceram antes.

onde ficaram as 298 pausas humanas



implementação, onde os erros nascem: 0 pausas

0.47

REVISÃO GASTA VS. FALHA FUTURA
· 0.5 = CARA OU COROA

0.58

SÓ DA ESTRUTURA DA TAREFA ·
FRACO MAS REAL

Sinalizadas pela estrutura, sub-revisadas
por humanos: **47% falharam vs. 35%.**

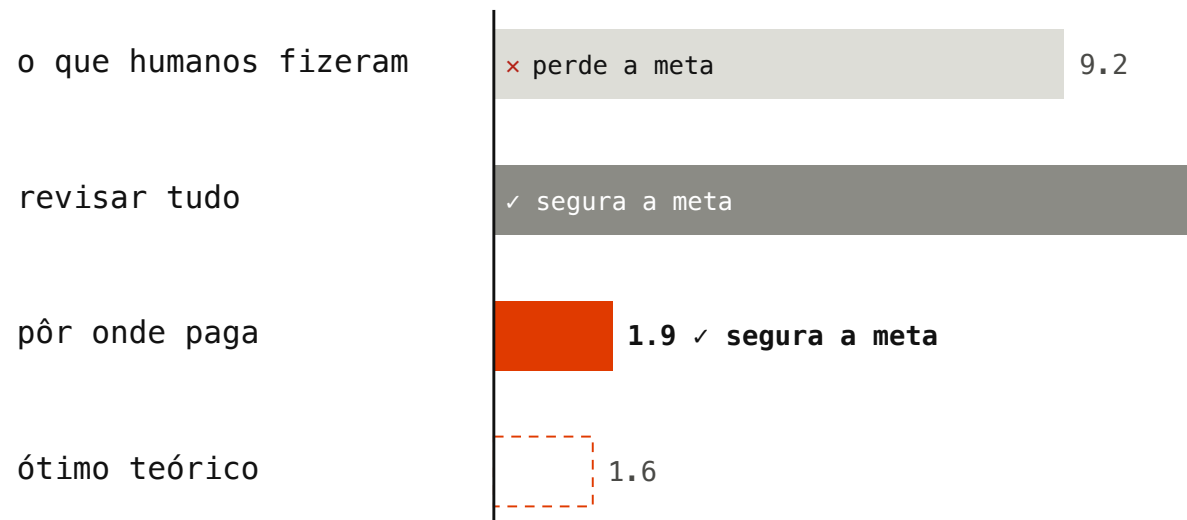
→ A atenção seguiu o calendário, e o risco ficou sem vigia.

PREVISTO revisão posta onde paga atinge a meta bem mais barato

CONFIRMADO · EM SIMULAÇÃO

Mesma qualidade, um sexto da revisão

custo de revisão para segurar a mesma meta · unidades relativas

 $\frac{1}{6}$

DO CUSTO DE REVISAR TUDO · 1.9 VS 11.4

Sob drift, planos fixos **perdem a meta**. O loop adaptativo **segura**: a política do planejador do Ato I, redecidindo conforme as estimativas mudam.

→ Um resultado de simulação. O próximo passo é um teste ao vivo.

O que os números desta noite podem e não podem dizer

MEDIDO

**bajulação 4x ·
posição da revisão**

Aconteceram.

Não provam causa.

CORRELAÇÃO

**tamanho ranqueia
risco**

Conhecido antes do
código.

**Não prova que mover a
revisão ajuda.**

SIMULAÇÃO

**custo $\frac{1}{6}$ · resultados
de deriva**

Previsto pelo modelo.

Ainda não implantado.

EXECUÇÕES AO VIVO

**O Ato IV soma um
quarto tipo**

Modelos reais, nota
mecânica, $n = 6$ por célula.

Evidência direcional.

→ **Cite cada número com seu rótulo.**

IV

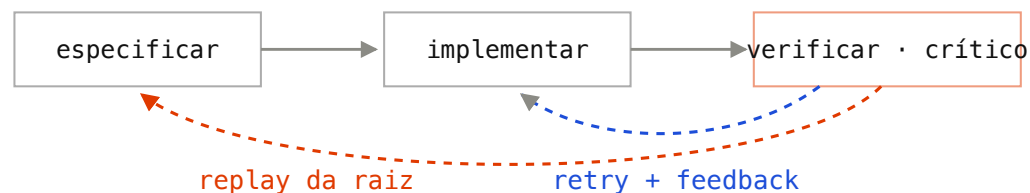
A PRÁTICA

Como aplicar review patterns em coding workflows?

Execuções em modelos reais.

Patterns: uma chain com feedback, e um fan-out com re-dispatch.

CHAIN · LangGraph · o implementador nunca vê o ticket



```
def gate(state) -> Command:
    verdict = ask(CRITIC, ...)
    if verdict.startswith("OK"):
        return Command(goto=END)
    if "DEFECT_SPEC" in verdict:
        return Command(goto="spec", update={"feedback": [verdict]})
    return Command(goto="implement", update={"feedback": [verdict]})
```

revisor: maritaca, outra família

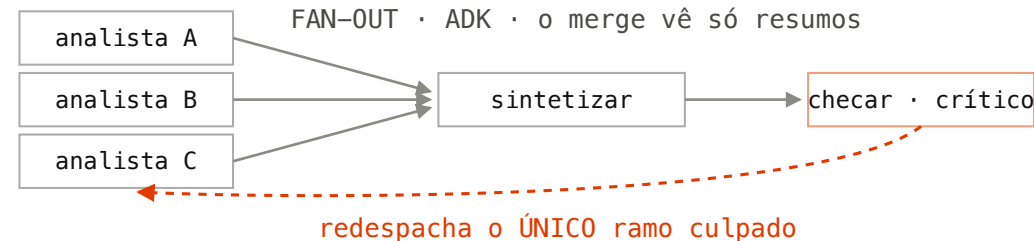
avaliador: juiz independente + checagens automáticas, fora do loop

gabarito: testes ocultos · gabaritos de prova

executores: gemini 3.5 / 3.6

o código completo no repo

FAN-OUT · ADK · o merge vê só resumos



```
async def _run_async_impl(self, ctx):
    async for ev in self.fan.run_async(ctx): yield ev # 3 analistas
    async for ev in self.synth.run_async(ctx): yield ev
    blamed = localize(ask(CRITIC, ...)) # revisar a junção
    async for ev in self.analysts[blamed].run_async(ctx):
        yield ev # repetir UM ramo e então unir de novo
```

O conserto foi o instrumento: críticos que produzem evidência

VEREDITO · UMA OPINIÃO

“Se o código satisfaz todos os critérios, responda exatamente: **OK**
Se violar um deles, responda:
DEFECT_CODE: ...”

PRECISÃO

0,00–0,20

EXECUTÁVEL · EVIDÊNCIA

“ **NÃO** julgue o código: escreva instruções **assert** que expressem os critérios. **Nós as executamos.**”

PRECISÃO

2x

mesmo modelo · ele deve escrever testes válidos

RUBRICA · EVIDÊNCIA + FEEDBACK

“Pontue **cada critério**: PASS ou FAIL. Não invente critérios.”

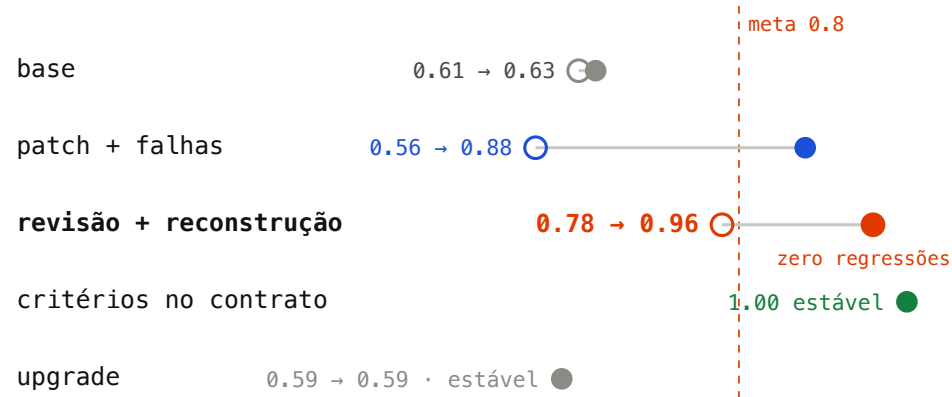
CONCORDÂNCIA DOS JUÍZES

0,85–0,97

critérios reprovados = mensagem de reparo

→ Um veredito é uma opinião. Uma asserção que executa, ou um critério pontuado, é evidência.

Com o instrumento certo, a revisão enfim paga



- **O upgrade não comprou nada:** a falha era informação perdida entre nós.
- **Revisão com feedback reconstruiu exatamente isso,** e a chain concordou baixinho (+0.02, zero regressões).
- **Critérios marcaram 1.00 em tudo:** contrato, lista de critérios e feedback são um artefato só.

n = 6 por célula · execuções únicas · modelos ao vivo

→ **O workflow diz o que comprar: revisão, profundidade de feedback ou um nó melhor.**

delegation-lab calcula qual alavanca

```
from google.adk import Agent
from minimal_oversight import analyze_pipeline

support = Agent(name="support",
    sub_agents=[triage, locate, patch, verify])
report = analyze_pipeline(support, p_min=0.90)
```

viável ou não

revisão por nó

ranking de mascaramento

agenda de recheragem

NÓ	HABILIDADE	O PLANO
triage	95%	pular
locate	70%	revisar 98% das execuções
patch	55% → 75%	upgrade + revisar 88%
verify	85%	revisar 49%

2.4 unidades de revisão põem todo nó $\geq 90\%$. “Revisar tudo” custa **4.0** e ainda perde: um nó de 55% não chega a 90% em cobertura nenhuma.

Aqui a matemática manda subir o nó; no fan-out ela mandou feedback.

→ O cockpit faz isso ao vivo: cole rastros ou um grafo, arraste a meta, veja o plano se mover.

Seis hábitos, ranqueados por efeito medido

1	Escreva os critérios antes de alguém agir	1.00 em toda célula ao vivo; contrato, lista de critérios e feedback são um artefato só	adicione um bloco de critérios a um template de tarefa
2	Registre dois números por tarefa	a revisão esconde a habilidade real, e pode inverter (mascaramento ao vivo 0.79)	adicione uma coluna pré-revisão
3	Faça críticos produzirem evidência	precisão de veredito 0.00 → 2×; juízes que dão nota critério a critério concordam 0.85–0.97	transforme um crítico em asserts ou em nota critério a critério
4	Declare o tamanho no planejamento	tamanho prevê dano 2× melhor que dificuldade	um campo de tamanho · divida o que for XL
5	Divida antes de delegar	algumas metas de qualidade são impossíveis em qualquer orçamento	faça a divisão para um pipeline
6	Re-estime em agenda fixa	cinco updates de modelo em dois meses mexeram toda habilidade	agende · mantenha auditorias aleatórias

→ Os três do topo são as principais alavancas.

O que você aprendeu esta noite

ENGENHARIA DE SOFTWARE

event sourcing

o log é a única verdade

design by contract

o pedido é fixado de
antemão

defesa em profundidade

checagens independentes
em camadas

PLANEJAMENTO DE IA

MDP fatorado

o mundo em variáveis
pequenas

RDDL

uma língua para MDPs
fatorados

alocação marginal

gaste cada olhada onde
compra mais

ENGENHARIA DE HARNESS

hooks

código em momentos fixos
do loop

adaptadores & sidecars

plugam qualquer harness
no log

playbooks & skills

passos fixos, modelo
dentro

ESTA TEORIA

supervisão mínima suficiente

a menor revisão que
mantém a qualidade

maskamento

a revisão esconde a
habilidade real

o pico de 75%

onde a revisão paga mais

profundidade de reparo

patch · retry · replay

→ Cada um destes está a uma busca de distância.

~~“Revisar tudo”~~

6x o custo pela mesma qualidade

~~“Confiar no pipeline verde”~~

a revisão esconde o agente: **bajulação de 1.8x**

~~“Vigiar o trabalho mais fraco”~~

a revisão paga mais perto de **75% de habilidade**

Autonomia é um **orçamento**, definido pelo pipeline inteiro; o modelo contribui um termo.

Cheque se é viável. Gaste onde paga. Recheque em agenda fixa.
A revisão se paga quando críticos produzem evidência.

paper

arxiv.org/abs/2606.15563

código

github.com/crbazevedo/delegation-lab

instalar

`pip install minimal-oversight`

eu

@crbazevedo

Obrigado. Perguntas?



APÊNDICE • PARA O Q&A

**Fórmulas • confiabilidade
dos juízes • estatísticas
de estrutura • referências
• história de campo •
anexos • partes puladas
• seis anos de modelos**

As fórmulas por trás do Ato II

$$\sigma_{\text{corr}} = \sigma_{\text{raw}} + (1 - \sigma_{\text{raw}}) \cdot b \cdot c \cdot f \cdot \text{olhou} \times \text{detectou} \times \text{corrigiu}$$

emendada (execuções ao vivo): –
 $\sigma_{\text{raw}} \cdot b \cdot (1 - p) \cdot p$ · reparos falso-positivos
 destroem trabalho correto

$$\alpha^*(x) = \min(\alpha_{\text{max}}, (\lambda/2) \cdot \sigma / (\sigma(1 - \sigma))) \cdot \text{pico} \approx \mathbf{0.75}$$

$$B_{\text{eff}} = C_{\text{op}} - p_{\text{min}} - \lambda \cdot H(W) \cdot \lambda \approx 0.02 / \text{bit}$$

$$T^*_{\text{auto}} = B_{\text{eff}} / \mu_{\text{eff}} \cdot \text{inclinação observada} - 0.99$$

$$K/N \geq (p_{\text{min}} - \sigma_{\text{raw}}) / [(1 - \sigma_{\text{raw}}) \cdot c \cdot f] \cdot \text{acima de } 100\%: \text{ revisão sozinha não atinge a meta}$$

$$\text{replay iff } w \cdot [f(1 - p)\bar{\sigma}^d - f_1^{up}] > \lambda \cdot \Delta C(d)$$

$$r_3(j) = \ell \cdot b_j c_j \cdot \sum_{m < j} \tilde{\pi}_m \prod (1 - b_g c_g e_g)$$

→ Cada quantidade é estimável a partir das duas colunas do log.

Mesmos agentes, forma melhor: 46% → 88%

antes · um agente · 12 ferramentas · chain de seis passos



46%

depois · três agentes · 4 ferramentas cada · emenda revisada por passagem



88%

- **Menos ferramentas por agente:** cada dobra nas escolhas taxa o buffer de autonomia ($\approx 2\%$ por bit).
- **Divida o mega-agente:** pedidos menores sobem a habilidade crua. É o achado do tamanho.
- **Revise nas emendas:** a passagem dá à olhada onde se segurar.

exemplo resolvido · passos 88% → 93% · detecção 80% · correção 85%

→ **Forma amigável à governança é escolha de projeto; aqui ela subiu a qualidade.**

Este grafo, avaliado: revise as junções, trave as chains fundas

117

MERGES · 14% DE TODOS OS NÓS

113

FAN-OUTS

65

DIAMONDS · 56% DOS MERGES

10

CHAIN MAIS FUNDA

PESADO EM MERGES → ALAVANCAGEM

117 olhadas (14% dos nós) ficam a jusante de todo ramo paralelo. **Revisar só as junções inspeciona a maior parte do trabalho conjunto do grafo.**

CHAINS FUNDAS → GATES, POR ARITMÉTICA

Profundidade 10 a 95%/passo entrega **60% cru**. Revisar o fim de toda execução ainda trava em **87%**, abaixo da meta. **Chains $\geq$ 5 pedem gates no meio.**

DIAMONDS → RISCO DE RECONCILIAÇÃO

56% dos merges juntam braços que precisam concordar: a forma de fan-in que previu dano. **Prioridade: junções → gates de chain → ramos amostrados → pare.**

→ **A estrutura diz onde olhar, antes de qualquer resultado chegar.**

Juízes LLM precisam ganhar confiança

CHECAGEM	VALOR
concordância ponderada, re-nota reservada κ_w	0.84
concordância entre quatro avaliadores (Fleiss κ)	0.61
dentro de um nível de dano	100%
juiz mais duro	reportado, sem correção
construções alternativas de rótulo	achados inalterados

- Dano pontuado 0–3 no campo “o que quebrou” de cada retrospectiva.
- 815 retrospectivas · re-nota reservada de 120 tarefas por outros juízes.
- Contagens de menção medem ruído de processo; o juiz pontua o dano.

→ **Confiabilidade de juiz é um número que você mede e reporta.**

A estrutura recuperada, e onde a habilidade do modelo importa

16

FAMÍLIAS DE PAPEL (864
NÓS · 671 ARESTAS)

10

PROFUNDIDADE MÁXIMA DE
CHAIN

113

FAN-OUTS

117 · 65

MERGES · DIAMONDS

ENTRE MODELOS, SWE-BENCH (16 MODELOS)

inclinação habilidade→resolução por dificuldade:

fácil **0.91** · média **1.11** · difícil **0.73**

Upgrades de modelo compram mais nas **tarefas médias**.

O elo com habilidade média repousa numa premissa não testada.

→ Toda forma de workflow que a teoria assume ocorre em produção.

Referências & ponteiros

- **Azevedo (2026)** · Minimal Oversight · arXiv:2606.15563
- **delegation-lab** · toolkit + demos · github.com/crbazevedo/delegation-lab
- **Estudo empírico** · supervisão em rastros reais de agentes · em revisão, 2026
- **Cemri et al. (2025)** · por que sistemas multiagentes falham (MAST) · arXiv:2503.13657
- **Jimenez et al. (2024)** · SWE-bench · ICLR
- **Bowman et al. (2022)** · supervisão escalável · arXiv:2211.03540
- **Madras et al. (2018)** · learning to defer · NeurIPS
- **Geifman & El-Yaniv (2017)** · predição seletiva · NeurIPS
- **Zheng et al. (2023)** · LLM-as-a-judge · NeurIPS
- **van der Aalst (2016)** · Process Mining · Springer

→ **Lâminas + notas do apresentador vivem neste arquivo: aperte N.**

A execução que parou em silêncio

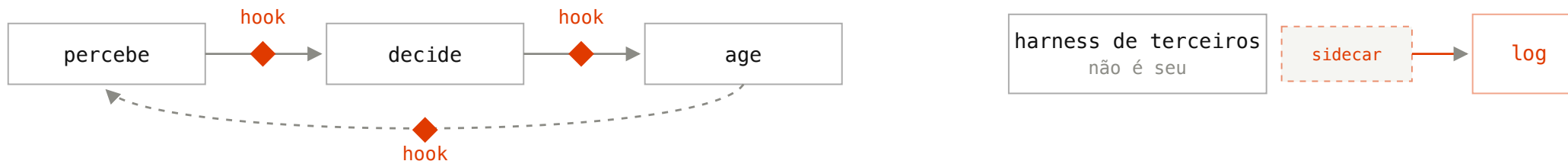
Uma execução autônoma longa pausou.
Sem crash. Sem erro.

O modelo vinha segurando a decisão de parar ou continuar, e escolheu **parar**, sem registrar nada.

- Parar-ou-continuar saiu do modelo, para regras fixas sobre o log.
- “Parar” não está no vocabulário do modelo; o fallback é trabalho ocioso útil.
- Só o kill switch do operador para o loop.

→ O modelo nunca deve decidir quando parar.

Como o plano de controle toca um agente



HOOKS

Código em momentos fixos do loop do agente: antes de uma chamada de ferramenta, depois da saída. **Pode bloquear: é código em vez de prompt.**

SKILLS

Conhecimento prático empacotado que o agente carrega sob demanda: procedimentos, checklists, regras de domínio.

ADAPTADORES & SIDECARS

Um processo companheiro acoplado a um harness que não é seu. Ele traduz os eventos daquele harness **para o log**. Mesma ideia dos sidecars do Kubernetes.

PLAYBOOKS

Sequências fixas de passos para tipos de tarefa conhecidos. O modelo improvisa **só dentro de um passo**.

→ O que precisa acontecer é código. O modelo preenche as lacunas de julgamento.

O resto do plano de controle, em quatro linhas

BLAST RADIUS

Ações ranqueadas de só-leitura → irreversível. Um workflow é tão arriscado quanto seu **passo mais arriscado**.

APROVAÇÕES DE USO ÚNICO

Ações perigosas consomem uma aprovação emitida para aquele **estado exato**.

CONTROLE DE ADMISSÃO

Planos recebem uma de cinco respostas graduadas: aceitar · reduzir · endurecer · adiar · rejeitar.

CONFIANÇA CONQUISTADA

A confiança é computada de resultados do log. A revisão **nunca desliga**; a taxa se move.

→ Um instinto em tudo: transformar julgamentos em comparações contra o log.

Seis anos de modelos, por família

OPENAI

- jun 20** GPT-3
texto atrás de API; humano lê tudo
- nov 22** ChatGPT
escala de massa em 5 dias
- mar 23** GPT-4
boa o bastante para pularem a revisão
- set 24** o1-preview
raciocínio oculto por trás da resposta
- abr 25** o3
ferramentas dentro do raciocínio
- ago 25** GPT-5
cadeias longas de ferramentas

ANTHROPIC

- mar 23** Claude 1
assistente HHH via API
- mar 24** Claude 3
família em camadas, com visão
- out 24** computer use
opera tela, cursor e teclado
- mai 25** Claude 4
horas de código autônomo
- set 25** Sonnet 4.5
execuções de 30 horas
- jun 26** Claude 5
salvaguardas por nível no produto

GOOGLE

- mar 23** Bard
chat ao lado da busca
- fev 24** Gemini 1.5
contexto de 1M de tokens
- dez 24** Gemini 2.0
modelo para a era agêntica
- mar 25** Gemini 2.5
orçamento de raciocínio exposto
- nov 25** Gemini 3
plataforma orientada a agentes
- mai 26** Gemini 3.5 Flash
agente pessoal 24/7 (Spark)

DEEPSEEK + ABERTOS

- jul 23** Llama 2
pesos abertos amplamente adotados
- dez 24** DeepSeek-V3
fronteira auto-hospedável
- jan 25** DeepSeek-R1
raciocínio aberto, licença MIT
- jul 25** Kimi K2
agêntico aberto, 1T parâmetros
- abr 26** DeepSeek-V4
contexto de 1M, aberto

0 ARCO **2020-22** texto lido por resposta · **2023** revisão pulada, código disparado · **2024** ferramentas, contexto de projeto, raciocínio oculto · **2025** agentes de horas, com permissões e auditoria · **2026** governança contínua de sistemas de longa duração

ECOSSISTEMA **jun 23** function calling · **nov 24** MCP · **fev 25** CLIs de código no terminal · **mar 25** horizonte METR dobrando em meses · **set 25** execuções de 30 h anunciados